

SMTP Fundamentals and Workflow

SMTP is the Internet standard for sending mail from a client or MTA to another mail server. It uses text commands (e.g. EHLO, MAIL FROM:<sender>, RCPT TO:<recipient>, DATA) in a request-response handshake. Common SMTP commands and responses (e.g. 250 OK, 354 Start mail input) control the transfer. An SMTP **submission** (usually port 587) is used by end-systems (mail clients) to send outbound mail, whereas port 25 is traditionally for MTA-to-MTA relaying. Port 465 (“smtps”) may be used for implicit TLS. Always prefer encryption: either connect with secure: true (TLS) or use STARTTLS on port 587 to upgrade to TLS, protecting credentials and message content

- **Ports & Authentication:**
 - *Port 25:* Default MTA relay port (often blocked by ISPs for abuse)
 - *Port 587 (submission):* Default for authenticated clients; use STARTTLS to encrypt
 - *Port 465 (SMTPS):* Legacy port for implicit TLS (commonly used with Gmail/SES).
 - *Auth:* SMTP servers require login credentials (username/password) or OAuth2 tokens for services like Gmail/SparkPost. All traffic should use TLS encryption (TLSv1.2+) to prevent eavesdropping.
- **Common Headers:** Each email has standard headers carrying metadata:
 - **From:** Sender’s name and address. (Mandatory)
 - **To:** Primary recipient(s). Optional CC/BCC fields.
 - **Subject:** Title of the message.
 - **Date:** Timestamp of sending (Mandatory)
 - **Reply-To:** Where replies should be directed (optional).
 - **Message-ID:** Unique identifier for the email.
 - **Received:** Added by each MTA hop to trace the route.
 - **Return-Path:** Bounce address for undeliverable mail (set by receiving MTA).

Properly setting From and configuring SPF/DKIM/DMARC records in DNS ensures recipients trust the email origin

- **Workflow (MTA vs MDA vs MUA):** In a typical email flow: the Mail User Agent (MUA, e.g. Outlook, webmail) hands the message to a local MTA (Mail Transfer Agent, e.g. Postfix, Sendmail) via SMTP. That MTA may relay the message through one or more intermediate MTAs over the Internet (also via SMTP). Finally the destination MTA hands the mail to a local Mail Delivery Agent (MDA, e.g. Dovecot, Procmail), which stores it in the recipient’s mailbox. The recipient’s MUA then retrieves it via IMAP/POP3. This flow is illustrated below:

graph LR

```
A[MUA (Client App)] -- SMTP --> B[MTA (Outgoing Server)]
B -- SMTP --> C[MTA (Destination Server)]
C -- Local Delivery --> D[MDA (Inbox Store)]
D -- IMAP/POP3 --> E[MUA (Recipient)]
```

Here each arrow denotes email transfer (SMTP or retrieval). Every time a server accepts the mail, it adds a **Received** header. This distributed MTA/MDA architecture is why SMTP is a “push” protocol, and the final retrieval (IMAP/POP3) is a “pull” by the mail client.

Eventbrite’s Email Delivery Practices

The eventbrite’s SMTP likely;

Layer	Technology
Application backend	Custom backend services
Email provider	Amazon SES / SendGrid / Mailgun
Delivery method	SMTP and/or Email APIs
Email authentication	SPF, DKIM, DMARC

Sample Next.js + TypeScript SMTP Email Sender

Below is an example of a Next.js API route (in **pages/api/send-email.ts**) using **Nodemailer** to send an email via SMTP. It demonstrates secure configuration, environment variables, error handling, and a curl test example.

```
// pages/api/send-email.ts
import type { NextApiRequest, NextApiResponse } from 'next';
import nodemailer from 'nodemailer';

// Ensure required SMTP env vars are set
const { SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, SMTP_FROM } = process.env;
if (!SMTP_HOST || !SMTP_PORT || !SMTP_USER || !SMTP_PASS || !SMTP_FROM) {
  throw new Error('Missing SMTP configuration in environment');
}

export default async function handler(req: NextApiRequest, res: NextApiResponse) {
  if (req.method !== 'POST') {
    res.setHeader('Allow', 'POST');
    return res.status(405).json({ error: 'Method Not Allowed' });
  }

  const { to, subject, text, html } = req.body;
  if (!to || !subject || (!text && !html)) {
    return res.status(400).json({ error: 'Missing email parameters' });
  }

  // Create a Nodemailer transporter using SMTP
  let transporter = nodemailer.createTransport({
    host: SMTP_HOST,
    port: Number(SMTP_PORT), // e.g. 587 for STARTTLS
    secure: SMTP_PORT === '465', // true for port 465, false for 587
    auth: {
```

```

    user: SMTP_USER,
    pass: SMTP_PASS,
  },
  // Require TLS to encrypt the connection
  requireTLS: true,
  tls: {
    // do not fail on invalid certificates if using a custom SMTP
    rejectUnauthorized: true
  }
});

try {
  // Send the email
  await transporter.sendMail({
    from: SMTP_FROM, // e.g. "Your Name <no-reply@example.com>"
    to: to,           // recipient or comma-separated list
    subject: subject,
    text: text,       // plain text body
    html: html        // optional HTML body
  });
  return res.status(200).json({ message: 'Email sent' });
} catch (error: any) {
  console.error('Error sending email:', error);
  return res.status(500).json({ error: 'Failed to send email' });
}
}

```

Environment Variables: Configure your SMTP credentials securely (e.g. in `.env.local` or Vercel Dashboard). For example:

```

SMTP_HOST=smtp.example.com
SMTP_PORT=587
SMTP_USER=your_smtp_username
SMTP_PASS=your_smtp_password
SMTP_FROM="Event Team <no-reply@example.com>"

```

Never commit these secrets to source control. On Vercel, set them in the project's Environment Variables panel.

Testing: You can test this endpoint with curl or Postman by sending a POST request with JSON. For example:

```

curl -X POST https://<your-domain>/api/send-email \
  -H 'Content-Type: application/json' \
  -d '{"to": "alice@example.com", "subject": "Hello", "text": "Hi there!"}'

```

A successful response should be HTTP 200 with `{ "message": "Email sent" }`.

Deployment Notes: On Vercel (serverless Next.js), creating a new transporter per request is fine, but avoid long-lived connections or global pooling. Use a reputable SMTP service (e.g. SendGrid, SES) to relay mail – Vercel itself recommends an external host for email sending.

Vercel's free tier may have limits on outbound connections, so ensure your provider and port are allowed. Also, enforce rate limits in your application (e.g. per-IP or per-user throttling) to prevent abuse – lacking rate limiting can lead to DoS or spam risks. Logging and error handling (as above) help diagnose issues if mail fails.

Security Best Practices: Always use TLS (as above) to encrypt SMTP. Validate inputs (to, subject) on the server to avoid header injection. Store credentials in env vars or a secret manager, not in code. Set up SPF, DKIM, and DMARC DNS records for your sending domain so recipients trust your messages. Monitor bounce and complaint webhooks from your provider. Finally, use email templates instead of sending user-supplied HTML to avoid XSS/malware in emails.

SMTP vs. Email API: Comparison Table

Aspect	SMTP (Traditional)	Email API (REST/Web API)
Ease of Setup	Simple: just host, port, username, password Supported by virtually all libraries/clients.	More complex: requires API key and coding (JSON or SDK). Each provider has its own API
Features & Analytics	Basic delivery only; no built-in templates or tracking by protocol. You'd add separate tracking tools	Rich features: templating, real-time tracking, analytics, webhooks, batch operations
Performance & Scalability	Slower for high volume: each email is sent in sequence with back-and-forth handshake. Limited throughput; harder to parallelize.	High throughput: optimized for bulk sends and parallel processing. Scales easily to millions of emails with low latency.
Deliverability & Reputation	Depends on your IP/domain reputation and config. Shared IPs or poor auth hurt deliverability. Must manage SPF/DKIM/DMARC yourself.	Providers often use warm, maintained IP pools and handle DKIM signing for you Usually more consistent inbox placement if set up correctly.
Security	Requires TLS configuration; if misconfigured, risks plain-text credentials SPF/DKIM/DMARC enforcement is manual.	Uses secured HTTPS with API keys/tokens. Providers handle TLS and authentication. Your domain still needs correct DNS records
Troubleshooting	SMTP gives step-by-step response codes (220, 250, 550 etc.), which can aid debugging. No unified dashboards.	APIs return structured JSON errors and often provide dashboards/logs. Easier to script and automate diagnostics.

This table summarizes key trade-offs. SMTP is universal and easy for simple use, but lacks advanced features and can be slower. Email APIs offer modern features and performance at the cost of needing provider integration Deliverability ultimately depends on proper authentication either way, but APIs often simplify this.

Sources: This analysis is based on SMTP standards and industry guides, Eventbrite's documentation and community forums and best-practice articles on Nodemailer and email APIs. All examples and recommendations here assume no specific SMTP provider; adapt the code above for services like SendGrid or Amazon SES by adjusting host/port and auth (e.g. SendGrid's SMTP user=apikey) as needed.